Edge cases (a template with the wrong variables) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — The idea, and the asymmetry
- Hint 2 — The plan, and almost the whole thing

> Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — The idea, and the asymmetry

Basic Version

A template has blanks in it, like `{question}`. When you call `.invoke(...)`, you have to give it a dictionary with a value for every blank.

This exercise asks: what happens if you forget one? And what happens if you give it an extra one it didn't ask for?

Try both, on purpose, and read the actual error message (or lack of one) carefully — don't guess.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

A `ChatPromptTemplate` knows, from parsing its own template string, exactly which variable names it needs. When `.invoke(...)` is called, it checks the dictionary you passed against that known set of names *before* doing anything else — including before any network call.

This means a missing variable fails fast, locally, with a `KeyError`-style message naming the exact missing variable — not a vague downstream failure. An *extra* variable your dictionary provides but the template doesn't use behaves differently: the template only looks for the keys it needs; an extra key that isn't referenced in the template string is silently ignored, not an error. This asymmetry matters — a missing variable is loud and cheap to catch; an extra, unused one is silent, which means a typo'd variable name you *meant* to use, but that doesn't match the template, won't be caught this way — it'll just be silently dropped.

Difference between Basic and Intermediate: Basic discovers a template's requirements by calling `.invoke(...)` and observing what happens. Intermediate catches the specific `KeyError` for the missing case and names precisely why the extra case is silently ignored — this is the depth the exercise's Solution is written at.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
build a template that needs one blank called question

test 1 - missing:
    call invoke with an empty dictionary
    catch the error, print it

test 2 - extra:
    call invoke with question filled in, plus an extra unused key
    print the result - does it error, or just work?
```

Here is almost the whole thing:

```
# prompt_template_practice.py
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_template("Answer: {question}")

try:
    prompt.invoke({})
except Exception as e:
    print("Missing:", type(e).__name__, e)
```

Expected output: something naming `KeyError` and `'question'`. Add the extra-variable test call yourself before checking the Solution.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
prompt = ChatPromptTemplate.from_template("Answer: {question}")

try:
    prompt.invoke({})
except KeyError as e:
    print(f"Missing variable -> KeyError: {e}")

result = prompt.invoke({"question": "What is LCEL?", "extra": "unused"})
print(f"Extra variable -> no error, result: {result}")
```

Notice the second call is not wrapped in `try/except` at all — because it's expected to succeed. If you find it doesn't, that's worth writing down as a real finding, not something to silently "fix" by assuming your first guess was right. Write both test calls yourself, then compare against the Solution.

Difference between Basic and Intermediate: both discover the missing/extra behavior by calling `.invoke(...)` once and reading what happens. Intermediate catches the specific `KeyError` class instead of a bare `Exception`, and splits both demonstrations into named, reusable functions.

Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution